

Name or Value? Cue Conflict Reveals When Chain of Thought Overrides Lexical Priors

Anonymous ACL submission

Abstract

Variable names that contradict a program’s behavior degrade the accuracy of language models, but a drop in accuracy does not indicate where inside the model the name interferes. Interpretability studies of multi-step arithmetic find that intermediate values are computed during chain-of-thought (CoT) generation; in those tasks, however, the name of a variable and its computed value never disagree. We introduce a cue-conflict version of such a task, in which the same problem is presented with congruent, neutral, and incongruent variable names, and evaluate eight models with three sets of few-shot examples, two of them on all five difficulty levels. Without CoT, the models use the value denoted by the name: a congruent name raises accuracy by up to 20 points, and an incongruent name is produced as the answer up to 20 points above chance. With CoT, we observe no effect of the name’s value at any level, with one exception. Linear probes indicate that the lure value is decodable at the name’s own tokens and is absent at the step where the chain writes the variable’s value, and activation patching localizes the no-CoT effect to the early layers at the equation that defines the name. Whether the chain states the value does not predict the size of the effect.

1 Introduction

Consider a short arithmetic problem written with named variables:

pen=1 + cup, cup=2 + 3; pen=?

The answer is 6. We then rename the second variable to the word two:

pen=1 + two, two=2 + 3; pen=?

The arithmetic is unchanged and the answer is still 6, but the name of the variable now denotes 2, while the variable itself holds the value 5. We refer to 2 as the *lure value* of the name and to 5 as its *true value*. If a model answers 2, or uses 2 in

a later step, the meaning of the name has reached the computation.

Placing two cues in conflict in this way is the design of the Stroop task (Stroop, 1935) and of the shape–texture experiments on vision models (Geirhos et al., 2019). We apply this design to multi-step arithmetic with variable assignments.

Motivation. Names that contradict the surrounding code degrade model performance (Le et al., 2025; Lam et al., 2025; Le et al., 2026), but these results do not indicate *where* the name interferes, since a drop in accuracy is consistent with a corrupted input, a corrupted computation, or a correct computation misread at the output. Kudo et al. (2026) show, with linear probes and activation patching on such arithmetic, that each variable’s value becomes linearly decodable only as the CoT writes it, but their variable names are single letters, so a decoded value of 5 can only originate from the computation. Our incongruent names supply a competing value, so that a probe’s output indicates which source the representation holds.

We construct matched sets of problems: the same arithmetic rendered in the naming conditions of Table 1 at the five difficulty levels of Table 2, solved with and without a written chain of thought, on eight models at level 3 and on two of them at every level. Without CoT, the models use the value denoted by the name: a congruent name adds up to 20 points of accuracy, and an incongruent name is produced as the answer up to 20 points above chance. With CoT, we observe no effect of the name’s value at any level, with one exception among the models we test. Probes indicate that the lure value is decodable at the name’s own tokens and is absent at the step that writes the variable’s value.

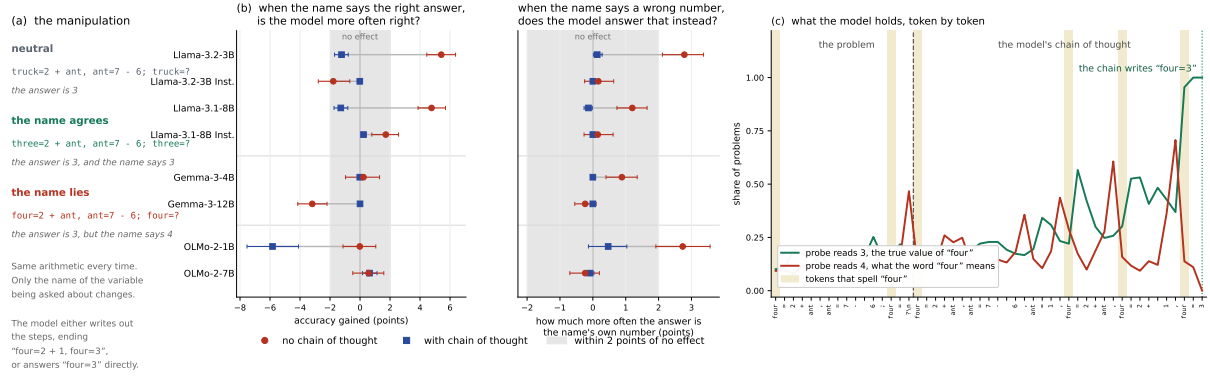


Figure 1: **(a)** The manipulation: one problem with three names for the queried variable, namely a neutral word, a number word denoting the answer, and a number word denoting a different value. The arithmetic and the answer are unchanged. **(b)** Effect of the name on the answer at level 3, one row per model, averaged over three sets of worked examples. Bars are 95% bootstrap intervals over problems, and the grey line joins the two regimes of one model. Left: accuracy gained when the name denotes the answer. Right: how much more often the model answers the number denoted by the name than the neutral twin produces that same digit. Without CoT (red) both effects are large. With CoT (blue), every model’s lure effect (right) falls inside the two-point equivalence margin fixed in advance; on the left, one model, OLMo-2-1B, loses accuracy when the queried variable carries a number word, whether or not it denotes the answer. **(c)** Probe predictions at each token of one problem. A probe trained only on neutral problems reads 4, the value denoted by the name four, through the problem and most of the chain, and the true value 3 appears only where the chain writes it (dotted line).

2 Setup

Task. We reuse the problems, difficulty levels and prompt format of Kudo et al. (2026). A problem is a list of equations over the digits 0–9 with + and −, ending in a query, and every variable’s value is itself a single digit. Three worked examples of the same level precede each problem, and Table 2 gives one problem per level. Level 3 is our main level.

Naming conditions. Each problem is generated once and rendered in the conditions of Table 1, where arithmetic, operators and digits are identical across a set and only names differ. In the *congruent* and *incongruent* conditions, exactly one variable, the *target*, is named by a number word, denoting the target’s true value in the former case and a different value, the *lure value*, in the latter. The target is either the queried or the intermediate variable, and we report the two cases separately, since they are different manipulations.

Generation rules. Only one variable per problem carries a number word, and the lure value differs from every variable’s value and from every digit in the problem, so that a lure answer cannot be correct for another reason. Names are single tokens in every context of the template under each model’s tokenizer, so twins have the same length and the same token positions (Appendix A). The

irrelevant lure condition places the number word on a distractor.

Regimes. In the *chain-of-thought* regime, the model writes the worked steps shown in Table 2 and then the answer. In the *direct* regime, it writes only pen=6. Our main comparison is between these two regimes.

Demonstration sets. The choice of the three worked examples affected accuracy more than any naming condition. Every behavioral result is therefore computed for three fixed demonstration sets and reported as their mean, and we count an effect as reliable only when its sign is the same in all three sets and a cluster bootstrap over matched problems excludes zero.

Probes. A linear probe reads a chosen variable’s value from the residual stream, one probe per position, layer and seed, following Kudo et al.. Probes are trained on neutral problems only, so that they never observe a number word in a name slot. Each is paired with a control probe trained on labels that are a fixed random function of the variable’s *name* (Hewitt and Liang, 2019); where the control also scores highly, the probe is reading the identity of the name rather than a value.

Patching. To test whether the name causes a behavior, we replace the activations at its tokens with

Condition	Problem	True
Letter	A=1 + B, B=2 + 3; A=?	–
Neutral	pen=1 + cup, cup=2 + 3; pen=?	–
Congruent	pen=1 + five, five=2 + 3; pen=?	5
Incongruent	pen=1 + two, two=2 + 3; pen=?	5
Irrelevant (level 4)	pen=1 + cup, cup=2 + 3, two=4 + 5; pen=?	9

Table 1: The naming conditions on one problem, whose answer is 6 in every row of the top block. The model writes either the chain pen=1 + two, two=2 + 3, two=5, pen=1 + two, pen=1 + 5, pen=6 or the answer pen=6 alone. *True* is the value of the variable carrying the number word, and *Lure* is the value that word denotes.

Level	Problem (neutral names)	Steps	Held	Dist.
1	pen=1 + cup, cup=2; pen=?	1	1	0
2	pen=2 + 3, cup=1 + pen; cup=?	2	0	0
3	pen=1 + cup, cup=2 + 3; pen=?	2	1	0
4	pen=1 + cup, cup=2 + 3, box=4 + 5; pen=?	2	1	1
5	pen=1 + cup, cup=2 + box, box=1 + 2; pen=?	3	2	0

Table 2: The five levels of Kudo et al. (2026). *Steps* is the number of additions or subtractions required, *Held* the number of variables that cannot be resolved when first read, and *Dist.* the number of equations the answer does not depend on. The chain restates each equation and writes each variable’s value in turn, as in cup=2 + 3, cup=5, pen=1 + cup, pen=1 + 5, pen=6. At level 1 the intermediate value is stated rather than computed, which serves as a no-computation control.

those of the matched twin and read the output again (Zhang and Nanda, 2024). Two controls accompany every intervention: replacing one neutral word with another bounds the damage of any replacement, and replacing one lure with another indicates whether the answer follows the name.

Models. Gemma-3-12B-it, Gemma-3-4B-it, Llama-3.1-8B, Llama-3.1-8B-Instruct, Llama-3.2-3B, Llama-3.2-3B-Instruct, OLMo-2-1B-Instruct, OLMo-2-7B-Instruct.

3 Results

Replication. On the letter condition, per-token probes trained on letters reproduce the positions reported by Kudo et al. for Llama-3.2-3B: the queried variable becomes decodable at equation 5 of the chain and the intermediate variable at equation 2, as in their Table 3. Our pre-CoT maxima, 30.4 and 47.9, exceed their 17.8 and 33.2.

Behavior. Table 3 reports accuracy and the pre-registered contrasts for each model and regime, averaged over three demonstration sets. Without CoT, the models use the value denoted by the name. A congruent name on the queried variable raises the accuracy of Llama-3.1-8B by +4.8 points at level 3 and by up to 20 points at level 5, and an incongruent name is answered +1.2 points above chance at level 3 and 20 points at level 5, where accuracy is near the floor (5 to 51 points across the three demonstration sets). With CoT, these contrasts are absent. An equivalence test with a margin of 2 points finds 33 of 34 CoT cells equivalent to zero, against 11 of 34 direct cells in which an effect is present. The one exception is the smallest model, OLMo-2-1B-Instruct, whose CoT solves 47% of level-3 problems with word names and 77% with letters. There, a lure on the intermediate variable adds +3.3 points [+2.5, +4.1].

Does the written value explain the effect of CoT? A natural explanation is that the chain states each variable’s value, and that the stated value replaces the value denoted by the name. We split the incongruent instances by whether the model’s own chain stated the true value of the target, and compare each half with the matched neutral twin under the same split. The raw lure rate differs considerably between the halves, but so does the rate at which the twin produces that digit, because a model whose chain has broken down produces an arbitrary digit. After subtracting this baseline, the excess is +0.1 points over 102 cells in which the value was written and +0.1 over 34 in which it was not. The exception survives the control: OLMo-2-1B-Instruct is lured +3.7 points [+2.2, +5.2] on the 1207 instances in which its chain wrote the correct value. Patching agrees: injecting the misleading name into a neutral run of that model, with a correct chain force-decoded, moves the answer to the lure in 2.7% of pairs at layer L5, against 0.4% for the word control at the same layer and 0.2% for every other model.

Irrelevant lures. At level 4 a number word on the distractor changes accuracy by at most 1.2 points. Without CoT it is nevertheless answered +2.1 to +2.3 points above the neutral baseline (2 of 2 cells reliable), so a number word naming an unused variable is also read as a value; with CoT the excess is +0.0 to +1.0 points (0 of 2 reliable).

Model	Regime	Neutral accuracy (%)	number word on the queried variable		number word on the intermediate variable	
			helps	lures	helps	lures
Gemma-3-12B-it	CoT	100 [100,100]	+0.0	+0.0	+0.0	+0.0
Gemma-3-12B-it	direct	78 [76,81]	-3.2*	-0.2	-3.7*	+0.4
Gemma-3-4B-it	CoT	100 [100,100]	-0.0	+0.0	-0.0	+0.0
Gemma-3-4B-it	direct	55 [48,59]	+0.2	+0.9*	+0.6	+0.3
Llama-3.1-8B-Instruct	CoT	100 [99,100]	+0.2	+0.0	+0.3	+0.0
Llama-3.1-8B-Instruct	direct	52 [45,65]	+1.7	+0.1	+5.2*	+0.2
Llama-3.1-8B	CoT	95 [86,100]	-1.3	-0.1	+3.8	-0.0
Llama-3.1-8B	direct	53 [41,62]	+4.8*	+1.2*	+3.8*	-0.0
Llama-3.2-3B-Instruct	CoT	100 [100,100]	-0.0	+0.0	-0.2	+0.0
Llama-3.2-3B-Instruct	direct	44 [40,50]	-1.8*	+0.2	+0.5	+0.5
Llama-3.2-3B	CoT	98 [97,100]	-1.2	+0.1	+0.9	-0.2*
Llama-3.2-3B	direct	22 [13,31]	+5.4*	+2.8*	+1.1	+1.8
OLMo-2-1B-Instruct	CoT	47 [44,51]	-5.8*	+0.5	+6.9*	+3.3*
OLMo-2-1B-Instruct	direct	34 [30,38]	-0.0	+2.7*	+3.0*	-0.2
OLMo-2-7B-Instruct	CoT	95 [93,97]	+0.6*	-0.1	-2.7	-0.0
OLMo-2-7B-Instruct	direct	54 [50,58]	+0.6	-0.2	+3.8*	-0.4

Table 3: Effect of the name’s value on the answer at level 3, averaged over three demonstration sets, with the range across those sets in brackets. *Helps* is the accuracy gained when the number word matches the variable’s value (congruent minus neutral, in points). *Lures* is how much more often the model answers the number word’s own value than it would by chance (incongruent minus the rate at which the neutral twin gives that same digit). * marks an effect whose sign is the same in all three demonstration sets and whose bootstrap interval excludes zero. Starred effects in the CoT rows are within two points, except for OLMo-2-1B-Instruct. Other levels are in Table 4.

Word class. One effect is not removed by CoT. At level 4, a number word on the queried variable costs Llama-3.2-3B 9.9 points (2–16 across demonstration sets) whether or not the denoted value agrees; the model then restates the wrong equation.

Probing results. Figure 1(c) shows the probe predictions at each token for Llama-3.2-3B at level 3 with CoT, and Figure 2 in the appendix the corresponding layer sweep. At P2, the end of the defining equation, the true value is partly computed (accuracy 0.49, selectivity 0.32) and the lure value is weakly decodable (0.06). At P3, the query, a probe reads the lure value in 32% of instances, but the control task also scores 0.97 there, so the probe is reading the identity of the name. At P4 and P5, the true value is decodable at 0.98 and the lure is at zero in every condition and level. At level 4 a number word on a computed variable has a lure mass of 0.004 at the answer, against 0.118 for one on the distractor, whose own value is not decodable (accuracy 0.04).

Causal intervention. Figure 3 in the appendix shows, for the direct regime, the proportion of lure errors removed by patching the tokens of the name from the matched neutral twin, by layer,

with both controls. For Llama-3.2-3B, patching all layers removes 52% of the lure errors and restores the margin between the true and lure values (normalized logit difference 1.00), the alternative-lure patch is followed in 9% of cases, and the span grid places the effect at the equation that defines the name, where patching layers 0–7 alone removes up to 69% of the errors. Because the word control changes 22% of correct answers at this floor accuracy, we report localization rather than recovery. For Llama-3.1-8B, whose no-CoT lure excess at level 3 is small, the neutral and alternative-lure patches remove lure errors equally, so the patching gives no evidence of a name-specific cause. With CoT, there are at most 1 lure errors in 2,000 instances to remove, and injection changes the answer in at most 0.2% of 500 pairs for every model except OLMo-2-1B-Instruct, reported above.

4 Related Work

Kudo et al. (2026) established when sub-answers are computed during CoT on clean arithmetic, and we add a cue conflict to that setting. Two studies apply the Stroop design to language models (Wang, 2026; Hu et al., 2026), but both measure a single next token, without a chain of thought. Faithfulness has been measured externally by perturbing prompts, chains, or parameters (Turpin et al., 2023; Lanham et al., 2023; Chen et al., 2025; Tutek et al., 2025); we measure it internally under a cue whose two values are known. Since small models can copy the trailing number of the chain (Liu, 2026) and an output prior can override a correct chain (Prabhakar et al., 2024), we report a positional-copy control (Appendix B). Misleading identifiers degrade performance on real code (Le et al., 2026, 2025; Lam et al., 2025); our synthetic task allows the effect to be localized. Work on binding (Feng and Steinhardt, 2024; Wu et al., 2025; Gur-Arieh et al., 2026) asks how a value attaches to a name; we ask whether that attachment survives a competing value.

5 Conclusion

We introduced a cue-conflict arithmetic task to ask where a misleading variable name interferes. Without CoT, the value it denotes is used as the answer, and patching localizes this to the early layers at the equation defining the name. With CoT, we observe no effect at any level, except in one of eight models, where the name still moves the answer through

a correct chain at a single early layer. Whether the chain states the value does not predict the effect’s size.

Limitations

The task is synthetic arithmetic with short chains, so the results need not transfer to real code, which is the motivation for the task rather than a finding. The models are at most 12B parameters, and within-family comparisons indicate the direction of a trend only. The lures are English number words; digit-bearing names such as x2, other languages, and optimized lures remain future work. Linear probes can miss information that is encoded nonlinearly; the patching experiments address this in part, but hidden states carry mixed information and patching cannot exclude side effects, which is why we report two control patches. Finally, the CoT format writes the name beside the computed value at every step, which may itself assist recovery; the direct regime bounds this effect from the other side.

References

Yanda Chen et al. 2025. Reasoning models don’t always say what they think. *arXiv preprint arXiv:2505.05410*.

Jiahai Feng and Jacob Steinhardt. 2024. How do language models bind entities in context? In *International Conference on Learning Representations*.

Robert Geirhos, Patricia Rubisch, Claudio Michaelis, Matthias Bethge, Felix A. Wichmann, and Wieland Brendel. 2019. ImageNet-trained CNNs are biased towards texture; increasing shape bias improves accuracy and robustness. In *International Conference on Learning Representations*.

Yoav Gur-Arieh, Mor Geva, and Atticus Geiger. 2026. Mixing mechanisms: How language models retrieve bound entities in-context. In *International Conference on Learning Representations*. ArXiv:2510.06182.

John Hewitt and Percy Liang. 2019. Designing and interpreting probes with control tasks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*.

Xiaoyang Hu, Mike Angstadt, Shane Storks, Zan Huang, Aman Taxali, Alex Weigard, Richard L. Lewis, and Chandra Sripada. 2026. Conflict and congruency effects in large language models: In-weight and in-context competition in a verbal conflict task. *arXiv preprint arXiv:2608.11510*.

Keito Kudo, Yoichi Aoki, Tatsuki Kuribayashi, Shusaku Sone, Masaya Taniguchi, Ana Brassard, Keisuke Sakaguchi, and Kentaro Inui. 2026. LLMs faithfully and iteratively compute answers during CoT: A systematic analysis with multi-step arithmetics. In *Findings of the Association for Computational Linguistics: EACL 2026*, pages 1114–1153. Code: <https://github.com/keitokudo/faithful-cot-multistep-arithmetic>.

Man Ho Lam, Chaozheng Wang, Jen-tse Huang, and Michael R. Lyu. 2025. CodeCrash: Exposing LLM fragility to misleading natural language in code reasoning. In *Advances in Neural Information Processing Systems*, volume 38. ArXiv:2504.14119.

Tamera Lanham, Anna Chen, Ansh Radhakrishnan, Benoit Steiner, Carson Denison, Danny Hernandez, Dustin Li, Esin Durmus, Evan Hubinger, Jackson Kernion, Kamilė Lukošūtė, Karina Nguyen, Newton Cheng, Nicholas Joseph, Nicholas Schiefer, Oliver Rausch, Robin Larson, Sam McCandlish, Sandipan Kundu, and 11 others. 2023. Measuring faithfulness in chain-of-thought reasoning. *arXiv preprint arXiv:2307.13702*.

Cuong Chi Le, Minh V. T. Pham, Cuong Duc Van, Hoang N. Phan, Huy N. Phan, and Tien N. Nguyen. 2025. When names disappear: Revealing what LLMs actually understand about code. *arXiv preprint arXiv:2510.03178*.

Jack Le, Anh H. N. Nguyen, and Tien N. Nguyen. 2026. Do machines struggle where humans do? LLM and human comprehension of obfuscated code. *arXiv preprint arXiv:2606.31725*. Cite in the third person under [review].

Ming Liu. 2026. The readout shortcut: Positional number copying dominates arithmetic CoT readout in small language models. *arXiv preprint arXiv:2605.22870*.

Akshara Prabhakar, Thomas L. Griffiths, and R. Thomas McCoy. 2024. Deciphering the factors influencing the efficacy of chain-of-thought: Probability, memorization, and noisy reasoning. *arXiv preprint arXiv:2407.01687*. Venue to verify (likely Findings of EMNLP 2024).

J. Ridley Stroop. 1935. Studies of interference in serial verbal reactions. *Journal of Experimental Psychology*, 18(6):643–662.

Miles Turpin, Julian Michael, Ethan Perez, and Samuel Bowman. 2023. Language models don’t always say what they think: Unfaithful explanations in chain-of-thought prompting. In *Advances in Neural Information Processing Systems*, volume 36, pages 74952–74965.

Martin Tutek, Fateme Hashemi Chaleshtori, Ana Marasović, and Yonatan Belinkov. 2025. Measuring chain of thought faithfulness by unlearning reasoning steps. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*.

Han-yu Wang. 2026. Persistent priors, preserved targets: A Stroop-style paradigm for lexical override. *arXiv preprint arXiv:2606.07555*.

Yiwei Wu, Atticus Geiger, and Raphaël Millière. 2025. How do transformers learn variable binding in symbolic programs? In *Proceedings of the 42nd International Conference on Machine Learning*.

Fred Zhang and Neel Nanda. 2024. Towards best practices of activation patching in language models: Metrics and methods. In *International Conference on Learning Representations*.

A Tokenizer checks

Rule 4 is decided per model, not by eye. For each model we tokenize every candidate name in the five contexts of the template (line start, after a comma, after each operator, before the separator) and keep only words that are a single token in all five with no merging into a neighbour. For the Llama 3 tokenizer, all ten number words pass and 95 of 112 neutral candidates pass; the lists are released with the code. The Gemma and OLMo models were run on the same lists, and the run-time guard confirmed single-token names and identical positions for them. Binary operators carry a space on each side because – otherwise merges with a following word (1–two tokenizes as 1, –two), which would break the alignment between twins. With the released lists, every instance of a condition tokenizes to the same length with the same position indices, and the runner excludes and counts any instance that does not; no instance was excluded in any run reported here.

B Full sweeps and selectivity

Positional-copy control. Liu (2026) report that small models copy whichever number stands in the trailing slot of the chain. Pooling the 4073 CoT lure errors at the answer position over every model, level and demonstration set, the number immediately preceding the answer in the model’s own chain is the lure in 13% of them and the target’s true value in 11%. Neither dominates, so positional copying does not account for the CoT lure errors that remain.

For every (variable, position, layer, seed) we report probe accuracy for the true value, lure rate, lure mass and margin, alongside the accuracy of a control probe trained on labels that are a deterministic random function of the variable’s name (Hewitt and Liang, 2019). Selectivity is the difference. The query position is the one cell where

a high lure rate coincides with control accuracy near 1, which is why it is reported as name identity rather than as a represented value. Figure 2 gives the per-token sweep for the incongruent condition with CoT; the sweeps for the other conditions and the direct regime, with t^* , t_{eq}^* and the pre-/post-chain accuracies in Kudo et al.’s format, are released with the code.

C Additional levels, models and demonstration sets

Table 4 repeats Table 3 for the other four levels.

Model	Level	Regime	Neutral accuracy (%)	number word on the queried variable		number word on the intermediate variable	
				helps	lures	helps	lures
Llama-3.1-8B	1	CoT	99 [98,100]	+0.0	+0.0	+0.3	+0.0
Llama-3.1-8B	1	direct	74 [38,99]	+16.1*	+11.2*	+8.8*	+1.3
Llama-3.2-3B	1	CoT	97 [95,100]	+1.1	+0.1	-2.4	+0.3
Llama-3.2-3B	1	direct	46 [21,93]	+0.6	+5.9*	-3.9*	+2.6*
Llama-3.1-8B	2	CoT	100 [100,100]	-0.1	+0.0	-0.0	+0.0
Llama-3.1-8B	2	direct	36 [33,37]	+11.8*	+0.7	+2.6	+2.2
Llama-3.2-3B	2	CoT	94 [90,98]	+3.7	+0.5*	+1.0*	-0.1
Llama-3.2-3B	2	direct	29 [27,30]	+10.8*	+2.5*	+2.6*	+2.2
Llama-3.1-8B	4	CoT	88 [63,100]	+1.0	-0.0	-2.3	+0.2
Llama-3.1-8B	4	direct	28 [26,31]	+13.7*	+5.8	+6.0*	+0.2
Llama-3.2-3B	4	CoT	64 [26,85]	-10.2*	+0.7	-2.6*	+0.4
Llama-3.2-3B	4	direct	20 [16,23]	-1.6	+1.9	-0.2	+1.2*
Llama-3.1-8B	5	CoT	65 [43,100]	+1.2	-0.3	+0.1	-0.1
Llama-3.1-8B	5	direct	20 [13,26]	+20.2*	+20.5*	+0.9	+0.4
Llama-3.2-3B	5	CoT	69 [48,97]	-1.4	+0.1	-2.5	-0.1
Llama-3.2-3B	5	direct	12 [11,12]	+4.7*	+3.5*	+1.1*	+0.5

Table 4: As Table 3, for levels 1, 2, 4 and 5. Level 1 is the no-computation control: the intermediate variable’s value is stated rather than computed.

Per-set values, the pooled estimate and the claim rule are released with the code, together with the span-by-layer patching grids for all three sources, the level-4 irrelevant-lure control, the lure-distance analysis, the teacher-forced lure rates, and the error taxonomy behind the word-class effect.

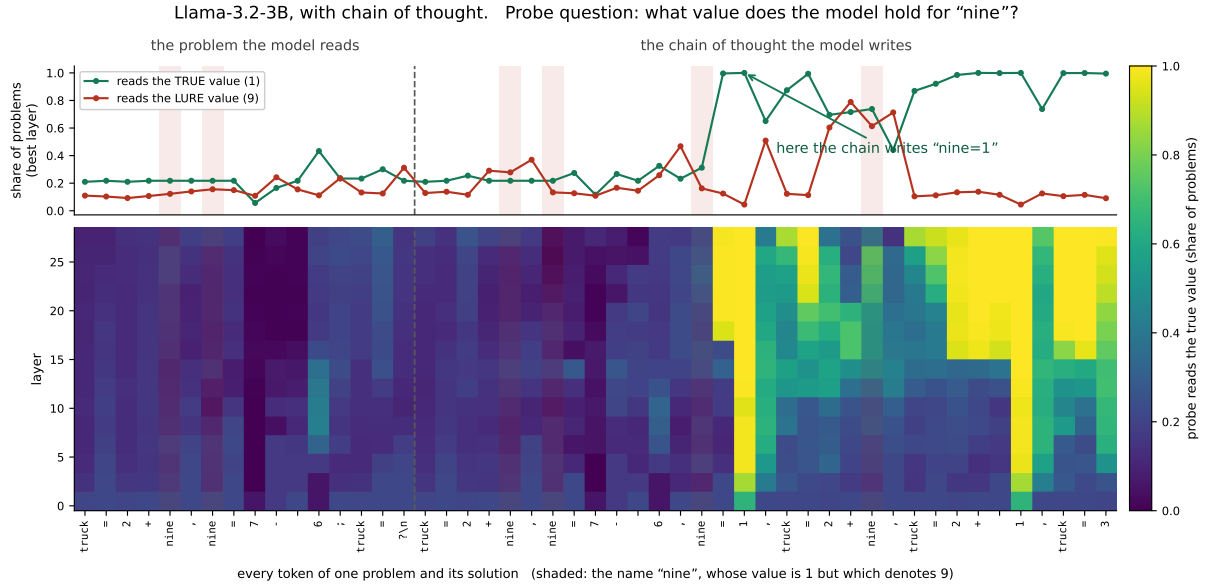


Figure 2: What the model holds for the variable whose name lies. The probe is asked, at every token, for the value of nine, a variable whose name denotes 9 but whose value is 1. **Top:** how often the probe’s answer is the true value (green) and how often it is the lure (red), taking the best layer at each token. **Bottom:** the same true-value accuracy for every layer. The problem is to the left of the dashed line and the model’s own chain of thought to the right; shaded columns are the tokens of the name itself. The lure is read only at those tokens and at the query, and it disappears the moment the chain writes `nine=1`, after which the true value is held for the rest of the solution. One problem’s tokens are shown; positions are the same for all 2,000.

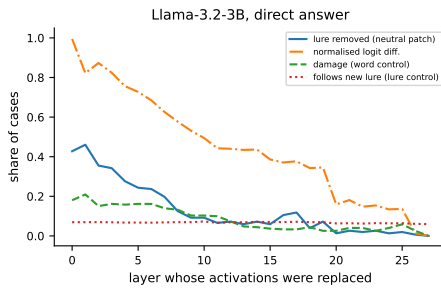


Figure 3: Lure errors removed by patching the name’s tokens from the neutral twin, by layer, direct regime, with the neutral-word control patch (damage) and the alternative-lure patch.